

PHASE 5

System Implementation and User Interface

Chapters 7–8: modules, routing, templates and screens

FITBUDDY • AI FITNESS PLAN GENERATOR • PROJECT REPORT

Chapter 7: System Implementation

This chapter describes how the design of Chapters 4 to 6 is realised in code. The description is organised by module and follows the project structure given in Appendix C. Code fragments are illustrative of the approach; where the exact implementation is not reproduced, the text says so. [Verify each fragment against the submitted source code]

7.1 FastAPI Application

The application is created as a single FastAPI instance in `main.py`. At start-up the module configures the Jinja2 template directory, mounts the static directory for CSS and JavaScript, loads environment variables, and ensures that the database tables exist. Routes are declared with decorators that bind an HTTP method and path to a handler function. Uvicorn serves the resulting ASGI application.

Request handling. For POST routes, FastAPI parses the request body, validates it against a Pydantic model and passes the validated object to the handler. The handler opens a database session, calls the relevant generator module, stores the result and returns a rendered template. Responses. Pages are returned as rendered HTML through `TemplateResponse`; the interactive documentation is returned by FastAPI itself.

Error handling. Three categories of failure are handled separately. Validation errors are detected before any processing and produce a clear message. Errors from the Gemini API (network failure, quota, invalid response) are caught in the generator modules and reported without writing partial data. Database errors cause the session to be rolled back. In every case the user receives an understandable message and previously stored data is left unchanged.

```
app = FastAPI(title="FitBuddy - AI Fitness Plan Generator")
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

@app.get("/")
def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})
```

The fragment above is illustrative of the initialisation pattern. [Replace with actual code if required]

7.2 Pydantic Schemas

Validation models are defined in `schemas.py`. `UserInput` describes the profile submitted from the form: user ID, name, age, weight, fitness goal and preferred intensity. `FeedbackRequest` describes a feedback submission: the user ID, the identifier of the plan concerned and the free-text feedback. Pydantic converts incoming values to the declared types and raises a validation error when a value is missing, of the wrong type or outside a declared constraint.

```
class UserInput(BaseModel):
    user_id: str
    name: str
    age: int = Field(..., ge=MIN_AGE, le=MAX_AGE)
    weight: float = Field(..., gt=0)
    goal: str
    intensity: str

class FeedbackRequest(BaseModel):
    user_id: str
    feedback: str = Field(..., min_length=1)
```

MIN_AGE and MAX_AGE stand for the limits defined in the implementation. [Insert exact validation rules from schemas.py] Centralising validation in these models means that route handlers can assume clean input, and that the same rules appear automatically in the API documentation.

7.3 Gemini Generator Module

gemini_generator.py is responsible for producing the seven-day workout plan. It exposes a function that accepts the validated profile, builds the workout prompt described in Section 6.6, calls the configured Gemini model through the google-genai client, and returns the parsed plan. Its responsibilities are limited to prompt construction, the model call and output checking; it neither reads from nor writes to the database.

- The client is created using the API key read from GOOGLE_API_KEY.
- The model name is read from GEMINI_WORKOUT_MODEL rather than being hard-coded.
- The response text is cleaned of stray formatting, parsed as JSON and checked for seven days.
- Exceptions are converted into a controlled error that the route handler can present to the user.

7.4 Gemini Flash Generator

gemini_flash_generator.py produces the nutrition and recovery tip. It follows the same pattern as the workout generator but uses the model named by GEMINI_NUTRITION_MODEL and a shorter prompt asking for brief, general guidance on hydration, balanced eating aligned with the goal, sleep and recovery. The output is plain text of a few sentences rather than structured JSON. Keeping this module separate allows the model, prompt and error behaviour to be changed without affecting workout generation.

7.5 Feedback Revision Module

updated_plan.py implements the Feedback Revision Engine. Its function receives the stored profile, the original plan and the feedback text, composes the revision prompt shown in Section 6.6, calls Gemini and validates the returned plan in the same way as the workout generator. It returns the revised plan to the route handler, which is the only component that stores it. Because the module receives the original plan as an input value and returns a new value, it has no means of modifying the original record, which supports requirement FR8.

7.6 Database Module

database.py creates the SQLAlchemy engine from DATABASE_URL, defines the session factory and declares the ORM models described in Chapter 5. Route handlers obtain a session through a dependency function that opens a session for the request and closes it afterwards. Helper functions or handler code perform the create, read and update operations for users, plans and feedback records. [Confirm whether helper functions are placed in database.py or in the route handlers]

```
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

```
def get_db(): db = SessionLocal() try: yield db finally: db.close()
```

The check_same_thread argument is specific to SQLite and is shown as a typical configuration. [Verify]

7.7 Routing

Table 7.1 lists the principal endpoints. The main workflow is the pair of POST routes; the GET routes provide entry, administration and documentation.

Table 7.1: Principal Endpoints

Method	Endpoint	Input	Processing	Output
GET	/	None	Renders the entry page	HTML form
POST	/generate-workout	UserInput fields	Validate; store user; generate plan and tip; store plan	Result page
POST	/submit-feedback	FeedbackRequest fields	Load profile and original plan; generate revision; store revision	Result page with original and revised plans
GET	/view-all-users	None (access-restricted)	Query users, plans, feedback and revisions	Administrative table page
GET	/docs	None	Generated by FastAPI from route definitions	Swagger UI

The route for generating a workout validates input first, so that invalid requests never reach Gemini. The feedback route additionally verifies that the referenced user and plan exist, and returns a clear message for an invalid user ID (test T15 in Chapter 9).

7.8 Frontend Templates

- index.html contains the fitness-profile form with fields for name, user ID, age, weight, goal and intensity, and a small script that disables the submit button and shows a progress message while the request is processed.
- result.html loops over the seven days of the stored plan, showing focus, duration and exercises with sets, repetitions and rest, followed by the nutrition tip and the feedback form. After a revision it shows the original and revised plans together.
- all_users.html presents the administrative view as a table of users with their plans, tips, feedback and revised plans. [Verify whether search and filtering (FR11) are implemented in this template, and adjust Section 3.1 accordingly]

Because the plan is stored as structured JSON, result.html can render it with Jinja2 loops rather than displaying unformatted text. User-supplied text such as feedback is escaped by Jinja2 autoescaping when displayed, which reduces the risk of script injection.

7.9 Styling

The interface is styled with a single stylesheet, static/css/style.css. Layout uses flexbox and CSS grid so that day cards arrange themselves in several columns on wide screens and a single column on phones. Media queries adjust font sizes, padding and table behaviour at tablet and mobile widths, and wide tables are placed in a horizontally scrollable container. Consistent spacing, sufficient colour contrast and clearly labelled form fields are used to support usability. [Insert breakpoints and colour choices used]

Chapter 8: User Interface

This chapter describes each screen of the application. No screenshots were available when this text was prepared, so each interface is described from the design and marked with a placeholder to be replaced by a real capture. The descriptions state intended behaviour and should be checked against the running application.

8.1 Home Page

The home page is the entry point. It presents the application name, a one-sentence explanation of its purpose, a visible notice that the tool provides general wellness guidance and not medical advice, and access to the profile form.

Figure 8.1: FitBuddy Home Page

Screenshot to be inserted

8.2 User Input Form

The form collects the six inputs listed in Table 3.6. Goal and intensity are chosen from labelled options, and numeric fields accept only numbers. If validation fails, the form is shown again with an error message. While the request is processed, the submit button is disabled and a progress message is displayed, since generation depends on the response time of the Gemini API.

Figure 8.2: Fitness Profile Form

Screenshot to be inserted

8.3 Generated Plan

The result page shows the profile summary, the seven-day plan as a sequence of day cards, and the nutrition and recovery tip. Each card shows the focus, duration and a list of exercises with sets, repetitions, rest and notes. Rest days are shown in the same format for consistency.

Figure 8.3: Generated Workout Plan

Screenshot to be inserted

8.4 Feedback Interface

Below the plan, a text area allows the user to describe what should change, for example shorter sessions or equipment constraints. Submitting the form sends the feedback to the revision route.

Figure 8.4: Feedback Form

Screenshot to be inserted

8.5 Revised Plan

After revision, the page shows the revised plan alongside or beneath the original so that the user can compare them. The original is labelled as such and remains unchanged.

Figure 8.5: Revised Workout Plan

Screenshot to be inserted

8.6 Admin Dashboard

The dashboard lists stored users and, for each, the original plan, nutrition tip, feedback and revised plan. Access is restricted as described in Section 4.7. Any personal data appearing in screenshots should be anonymised before inclusion in the report.

Figure 8.6: Admin Dashboard

Screenshot to be inserted

Figure 8.7: Swagger API Documentation at /docs

Screenshot to be inserted